

# ColuNere: Transposição Colunar Com Vigenere

Cauê Mateus Gonçalves Venturin Samonek<sup>1</sup>

João Eduardo Zangari Ambrosio<sup>1</sup>

<sup>1</sup>Departamento de Informática – Universidade Federal do Paraná (UFPR)

cmgvs23@inf.ufpr.br, jeza23@inf.ufpr.br

**Abstract.** Este trabalho apresenta a ColuNere, uma cifra construída pela combinação da Cifra de Vigenère com a Transposição Colunar. A implementação foi desenvolvida em C e comparada com o AES da OpenSSL e o RSA acessado pela biblioteca *cryptography*. Os experimentos consideram arquivos de diferentes tamanhos e medem o tempo médio de 5 execuções de cifragem e decifragem.

## 1. Informações Gerais

Este trabalho teve como objetivo implementar uma cifra própria a partir da combinação de duas técnicas de criptografia: uma cifra de substituição e uma cifra de transposição.

Foram escolhidas a Cifra de Vigenère, como método de substituição, e a Transposição Colunar, como método de transposição. As duas técnicas foram implementadas em linguagem C e combinadas em uma única cifra, denominada ColuNere.

Além do desenvolvimento da cifra, o trabalho compara seu desempenho com o AES-256-CBC executado pelo comando da OpenSSL e com o RSA de 2048 bits executado pela biblioteca *cryptography*. A comparação considera o tempo das operações de cifragem e decifragem para diferentes tamanhos de entrada.

## 2. Algoritmos

### 2.1. Cifra de Vigenère

A Cifra de Vigenère foi escolhida como componente de substituição. Ela utiliza uma chave textual repetida ao longo da mensagem para determinar o deslocamento aplicado a cada letra. Dessa forma, o deslocamento varia conforme a posição do caractere na chave.

Na implementação, as letras são representadas por valores entre 0 e 25. A cifragem pode ser expressa por

$$C_i = (P_i + K_i) \bmod 26, \quad (1)$$

enquanto a decifragem é dada por

$$P_i = (C_i - K_i + 26) \bmod 26. \quad (2)$$

Caracteres que não são letras são preservados. A chave avança apenas quando uma letra é processada, mantendo o mesmo comportamento na cifragem e na decifragem. Como o texto é percorrido uma única vez, essa etapa possui complexidade linear e baixo custo por caractere.

## 2.2. Transposição Colunar

Na Transposição Colunar, os caracteres são organizados em uma matriz com um número previamente definido de colunas. Durante a cifragem, o texto é escrito por linhas e lido por colunas. Na decifragem, o procedimento inverso recupera a ordem original.

Quando o tamanho do texto não é múltiplo do número de colunas, a implementação adiciona preenchimento ao final da entrada. Esse preenchimento é removido após a decifragem.

Embora o algoritmo tenha complexidade linear, a transposição exige a cópia e a reorganização completa dos dados. Para reduzir o custo de acessos distantes na memória, a implementação percorre a matriz em blocos. Essa estratégia melhora a localidade de referência e tende a aproveitar melhor a memória cache do processador.

## 2.3. Cifra ColuNere

A ColuNere aplica primeiro a Transposição Colunar e depois a Cifra de Vigenère. Na decifragem, a ordem é invertida: Vigenère é desfeita antes da transposição. Essa inversão é necessária porque cada transformação deve ser desfeita na ordem oposta àquela em que foi aplicada.

Cada entrada é, portanto, percorrida mais de uma vez e requer uma representação intermediária. O custo total corresponde à soma das duas etapas, sendo a Transposição Colunar a parte que realiza a maior movimentação de memória. Nos experimentos foi utilizada a configuração com quatro colunas.

## 2.4. AES

O AES é uma cifra simétrica, e dentre suas muitas variações, neste trabalho foi utilizado o AES com chave de 256 bits no modo CBC, por meio do comando `openssl enc`.

A OpenSSL possui implementações otimizadas para diferentes arquiteturas e pode utilizar instruções específicas do processador. O equipamento utilizado oferece suporte às instruções AES, o que reduz significativamente o custo das operações internas. Por esse motivo, mesmo realizando transformações criptográficas mais complexas, o AES pode superar uma implementação comum ou até otimizada escrita em C.

O vetor de inicialização foi mantido fixo durante os experimentos para que todas as execuções utilizassem os mesmos parâmetros. Essa decisão simplifica a comparação de desempenho, mas não deve ser reproduzida em uma aplicação real, na qual o IV não deve ser reutilizado com a mesma chave.

## 2.5. RSA

O RSA é um algoritmo assimétrico baseado em operações com inteiros grandes. A chave pública é usada na cifragem e a chave privada é usada na decifragem. Neste trabalho foi utilizado um par de chaves RSA de 2048 bits armazenado em arquivos PEM.

Uma operação RSA não aceita uma entrada de tamanho arbitrário. Com uma chave de 2048 bits e o padding PKCS#1 v1.5, cada operação pode cifrar no máximo 245 bytes e produz um bloco de 256 bytes. Por isso, o arquivo é dividido em blocos de 245 bytes durante a cifragem. Na decifragem, os blocos cifrados são lidos em grupos de 256 bytes e reunidos novamente.

Cada bloco é processado pela biblioteca Python `cryptography`, que usa o backend criptográfico disponível no sistema. O RSA é executado diretamente no processo principal, que carrega a chave uma vez por operação e processa o arquivo inteiro.

A decifragem RSA é mais lenta porque a cifragem utiliza normalmente o expoente público pequeno 65537, escolhido para reduzir a quantidade de multiplicações modulares. A decifragem utiliza o expoente privado, que possui tamanho muito maior e exige uma exponenciação modular mais custosa para cada bloco.

## 2.6. Aspectos da Implementação

A implementação da Transposição Colunar trata o texto como uma matriz linear armazenada em memória. Uma transposição direta percorre posições distantes ao alternar entre linhas e colunas, o que pode causar mais falhas de cache. Para reduzir esse efeito, os laços foram divididos em blocos de  $32 \times 32$  posições. Cada bloco pequeno tende a permanecer na cache enquanto é processado, melhorando a localidade espacial e temporal dos acessos. Essa técnica, também conhecida como *blocking* ou *tiling*, não altera a complexidade  $O(n)$  do algoritmo, mas reduz parte do custo de movimentação de memória.

O preenchimento da matriz é calculado antes da transposição. O programa aloca o vetor final uma única vez, copia o texto com `memcpy` e preenche somente a região restante com `memset`. Dessa forma, não ocorre uma realocação a cada caractere. Na decifragem, o mesmo procedimento de transposição é reutilizado com as dimensões invertidas, evitando a manutenção de duas versões independentes do laço principal.

A etapa de Vigenère também percorre a entrada apenas uma vez. O índice da chave é incrementado e reiniciado quando alcança seu comprimento, evitando uma operação de resto para localizar a chave em cada iteração. Caracteres que não são letras são copiados diretamente e não avançam esse índice. Para caracteres ASCII alfabéticos, a operação binária  $| \cdot 32$  normaliza a letra da chave para minúscula antes do cálculo do deslocamento entre 0 e 25.

A ColuNere lê o arquivo completo para a memória e mantém um vetor intermediário entre a transposição e Vigenère. Isso simplifica a composição das duas cifras e mantém os dados em regiões contíguas, mas faz o consumo de memória crescer linearmente com o arquivo. A saída de cada etapa é alocada uma vez e liberada logo depois de deixar de ser necessária.

No RSA, uma chamada separada ao comando `openssl pkeyutl` para cada bloco adicionaria milhares de criações de processo aos arquivos maiores. Por isso, o RSA é executado no processo principal. O arquivo PEM é lido, a chave é desserializada uma vez por operação e o mesmo objeto da biblioteca `cryptography` é reutilizado em todos os blocos. As exponenciações RSA são executadas pelo backend nativo da biblioteca, enquanto o Python controla a leitura e a escrita.

O AES é executado pelo comando `openssl enc`. O próprio comando realiza o processamento contínuo da entrada, sem que o programa Python precise separar o conteúdo em blocos. A chave textual do experimento é convertida para hexadecimal e completada até 32 bytes, enquanto o IV é fixo e possui 16 bytes, essa preparação fornece os tamanhos exigidos pelo AES-256-CBC apesar de não ser recomendado em um ambiente real.

## 2.7. Finalidade das Cifras

Vigenère e Transposição Colunar possuem principalmente valor histórico e didático. Elas ajudam a compreender os conceitos de substituição e transposição, mas não são consideradas seguras contra métodos modernos de criptoanálise. A ColuNere possui a mesma finalidade experimental, pois combina essas duas técnicas para permitir o estudo de sua implementação e de seu desempenho.

O AES é adequado para cifrar grandes quantidades de dados. Por ser uma cifra simétrica, utiliza a mesma chave na cifragem e na decifragem e consegue processar arquivos, discos e comunicações de forma eficiente. Entre os algoritmos comparados, ele é o que representa o uso normal para proteção do conteúdo de um arquivo.

O RSA é normalmente utilizado para proteger pequenas informações, como chaves criptográficas, e para produzir assinaturas digitais. RSA não foi desenvolvido para cifrar arquivos inteiros: cada operação aceita poucos bytes, tem custo elevado e aumenta o tamanho do resultado. Em sistemas reais, RSA costuma proteger uma chave de sessão, enquanto uma cifra simétrica processa o conteúdo. Neste trabalho, o arquivo é dividido em blocos e cifrado diretamente com RSA apenas para permitir a comparação experimental solicitada.

## 3. Metodologia Experimental

Foram utilizados 21 arquivos de texto obtidos do Projeto Gutenberg [1]. Os tamanhos variaram de 248 bytes a 3 MB, divididos em cinco faixas: até 1 KB, de 1 KB a 10 KB, de 10 KB a 100 KB, de 100 KB a 1 MB e acima de 1 MB.

Para cada arquivo foram executadas a cifragem e a decifragem com ColuNere, RSA e AES. Cada operação foi repetida cinco vezes e o resultado apresentado corresponde à média aritmética. A medição foi realizada com `perf_counter_ns`, envolvendo a invocação do processo e o acesso aos arquivos. Portanto, o tempo inclui tanto o processamento criptográfico quanto o custo de inicialização do executável e de entrada e saída.

O custo fixo de subprocesso existe na ColuNere, que inicia o binário desenvolvido em C, e no AES, que inicia o comando da OpenSSL. O RSA utiliza a biblioteca `cryptography` diretamente no processo principal e não possui esse mesmo custo. Para entradas pequenas, o tempo de inicialização representa uma parte significativa da medição e pode ser maior que o tempo gasto na cifra em si.

Por esse motivo, o AES pode aparecer mais lento que a ColuNere nos menores arquivos. A ColuNere executa poucas operações simples, enquanto o comando AES precisa carregar e inicializar a OpenSSL antes de processar a entrada. Conforme o arquivo cresce, esse custo inicial é diluído. Nesse ponto, as otimizações da OpenSSL e as instruções de hardware para AES passam a dominar, tornando o AES mais rápido no processamento dos dados.

Depois de cada conjunto de repetições, o arquivo decifrado foi comparado integralmente com o original através de `assert`, logo, a execução correta ocorre somente quando os arquivos são idênticos.

A geração do par de chaves RSA e a compilação da ColuNere ocorrem antes das medições e, portanto, não fazem parte dos tempos apresentados. Já a abertura dos

arquivos, o carregamento da chave RSA, a criação dos subprocessos da ColuNere e do AES e a gravação das saídas estão dentro da região medida. Os arquivos são processados em ordem crescente de tamanho e cada repetição sobrescreve a saída anterior, evitando acumular resultados entre execuções.

Os testes foram executados no seguinte ambiente:

- Processador: Intel Core i5-1135G7, 4 núcleos e 8 threads;
- Memória RAM: 8 GB;
- Sistema operacional: Linux Mint 22.3;
- Arquitetura: x86\_64;
- Compilador: GCC 13.3.0;
- Bibliotecas criptográficas: OpenSSL 3.0.13 e `cryptography`.

## 4. Resultados

As Figuras 1 e 3 apresentam os três algoritmos. Como o RSA puro possui tempos muito superiores, as Figuras 2 e 4 mostram apenas ColuNere e AES, permitindo observar melhor a diferença entre essas curvas.

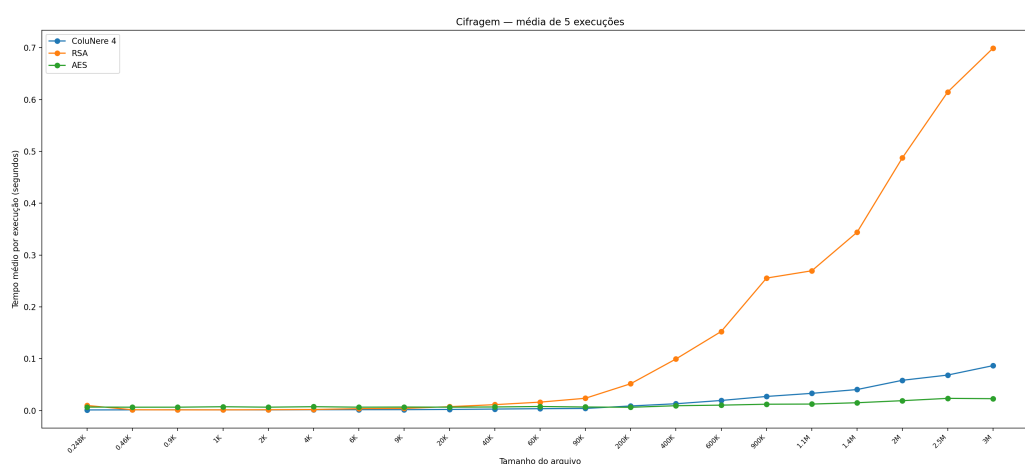


Figura 1. Tempo médio de cifragem com todos os algoritmos.

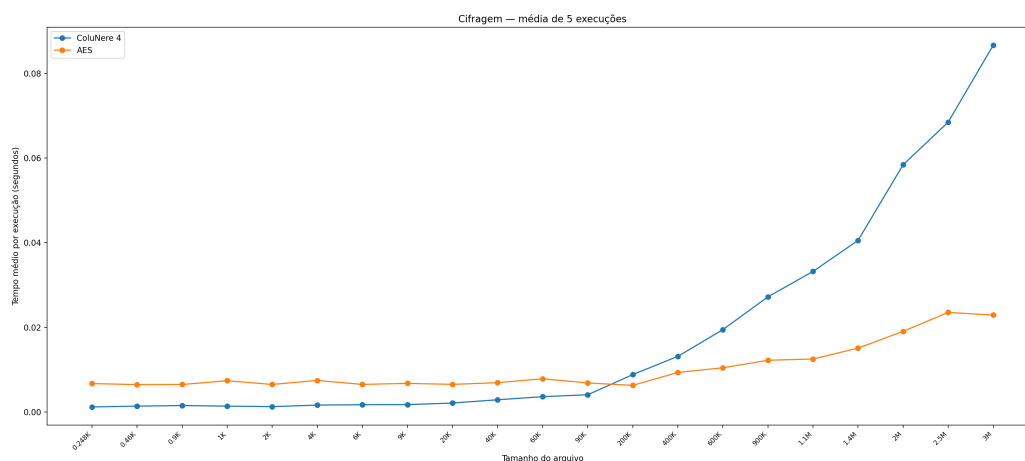
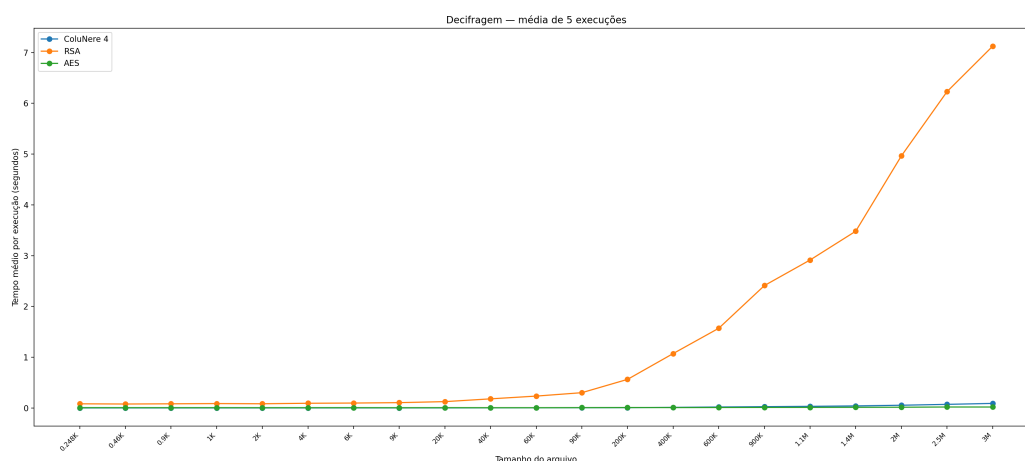
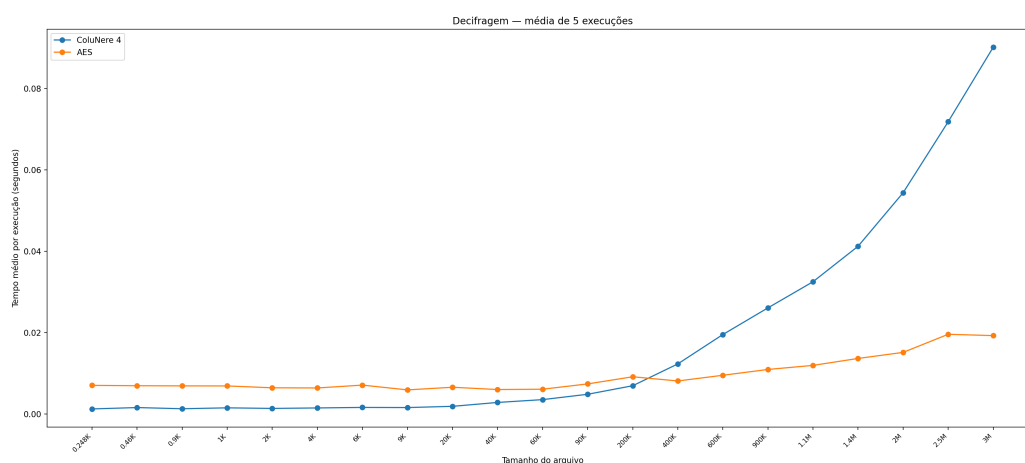


Figura 2. Tempo médio de cifragem com ColuNere e AES.



**Figura 3. Tempo médio de decifragem com todos os algoritmos.**



**Figura 4. Tempo médio de decifragem com ColuNere e AES.**

Para o arquivo de 3 MB, a ColuNere levou aproximadamente 0,0867 segundo para cifrar e 0,0901 segundo para decifrar. O AES levou 0,0229 e 0,0193 segundo, respectivamente. O RSA puro apresentou os maiores tempos: 0,6992 segundo na cifragem e 7,1250 segundos na decifragem.

Nos arquivos menores, a ColuNere aparece mais rápida que o AES porque o custo fixo de inicialização do subprocesso representa a maior parte do tempo. Com o aumento da entrada, esse custo é diluído e o AES passa a aproveitar suas otimizações e instruções de hardware. Em 3 MB, o AES foi aproximadamente 3,8 vezes mais rápido na cifragem e 4,7 vezes mais rápido na decifragem que a ColuNere.

O RSA puro é especialmente lento na decifragem, pois cada bloco exige uma operação com a chave privada. Esse resultado confirma que RSA não é adequado para processar diretamente o conteúdo completo de arquivos.

## 5. Limitações

Os três caminhos possuem overheads diferentes: o RSA é chamado diretamente como biblioteca, enquanto AES e ColuNere iniciam processos externos. Além disso, o cache do sistema operacional, o estado do processador, processos concorrentes e a ordem fixa dos testes podem causar pequenas variações.

Além disso, a ColuNere é uma combinação didática de cifras clássicas. Vigenère e Transposição Colunar não oferecem segurança adequada contra técnicas modernas de análise criptográfica. A comparação realizada neste trabalho é de desempenho e comportamento de implementação, não de nível de segurança.

## 6. Conclusão

O trabalho permitiu implementar e integrar duas técnicas clássicas em uma cifra funcional. A ordem inversa das operações de decifragem recuperou corretamente todos os arquivos testados, confirmando o funcionamento da ColuNere.

O AES processa o arquivo com uma chave simétrica, enquanto o RSA usa diretamente o par de chaves pública e privada sobre blocos de tamanho limitado.

Os resultados mostraram que o AES supera a ColuNere conforme o tamanho da entrada aumenta. O RSA puro apresentou o maior custo, principalmente na decifragem, confirmando que sua finalidade prática não é processar arquivos inteiros. Todos os arquivos decifrados foram comparados com os originais e recuperados corretamente.

## Referências

- [1] Project Gutenberg. *Free eBooks*. <https://www.gutenberg.org/>.